



STL 與基礎資料結構入門

S11459002 許哲豪

連結:<https://github.com/jhao-dev/Student-Information-Management-System-SIMS->



一、學習目標

- 1.學習如何使用 `std::vector` 動態管理自定義結構 `Student`
- 2.實作 `std::sort` 並結合 `Lambda` 運算式 進行自定義排序
- 3.透過 `cin.clear()` 與 `numeric_limits` 處理輸入異常流
- 4.實踐模組化設計： 將功能拆解為新增、查詢、排序與統計等獨立函式

二、Template 與 STL 的關係

簡單來說，Template 是一種技術，讓我們寫一次程式碼就能套用到不同種類的資料。像是我們作業裡的 `vector<Student>`，那個 `vector` 本身就是一個 Template，它不只可以裝整數（`int`）或字串（`string`），也可以裝我們自己定義的 `Student` 結構。

而 STL就是把這些常用的 Template 通通打包好給我們用。像是我們要排成績時，直接呼叫 STL 裡的 `sort` 函式，再加上一段簡單的比較邏輯（`Lambda`）就能搞定，不用自己在那邊辛苦地從頭寫排序演算法。

所以這兩者的關係就是：Template 提供了「不用重複寫程式碼」的技術，而 STL則是利用這種技術幫我們準備好的「懶人包」。對我們學生來說，這讓我們在寫像「學生管理系統」這種專題時，可以把精力花在處理功能邏輯（如新增、查詢、統計），而不用去煩惱底層的資料要怎麼存、怎麼排。

三、常見 STL 容器用途整理

1. `std::vector` (動態陣列) —— 專案中的主力

- 用途：就像一個可以自動伸縮的陣列。
- 為什麼我們用它：因為學生的資料是一筆接一筆存進去的，而且我們需要「列出所有學生」和「依成績排序」。vector 在記憶體中是連續排列的，這讓 `std::sort` 跑起來非常快。
- 缺點：如果要在中間刪除或插入資料，後面的資料都要移動，會比較慢。

三、常見 STL 容器用途整理

2. `std::map` (關聯式容器 / 字典) —— 最適合學號查詢

- 用途： 成對儲存資料 (Key-Value)，像字典一樣可以用關鍵字直接查內容。
- 適合場景： 如果我們的專案強調「快速搜尋學號」，用 `map<string, Student>` 會比 `vector` 快很多。在 `vector` 裡找學號需要從頭到尾巡一遍，但 `map` 可以像查字典一樣，直接跳到該學號的位置。

三、常見 STL 容器用途整理

3. `std::list` (鏈結串列) —— 頻繁增刪的好幫手

- 用途：像是一列火車，每一節車廂都連著下一節。
- 適合場景：如果這個管理系統需要頻繁地在名單中間「插隊」或「開除學生（刪除資料）」，用 `list` 會比 `vector` 有效率，因為它只要把車廂間的鉤子重新連一下就好，不用像 `vector` 要搬移所有資料。

四、第2週小專題設計說明

這個是要設計一個功能完整的學生資料管理系統，主要利用 C++ 的 `struct` 定義資料欄位，並搭配 STL 的 `vector` 容器來動態儲存學生資訊。程式架構以選單為核心，實作了資料新增、列表、學號查詢、成績排序與數據統計等五大功能，其中最關鍵的是運用了 `std::sort` 結合 Lambda 運算式來達成成績由高到低的排名，並加入輸入防錯機制，確保系統在處理使用者輸入時不會發生錯誤。

五、主要程式架構與重要程式片段

1. 資料封裝與儲存 (Data Structure)

這部分定義了系統的核心資料模型。

```
struct Student {  
    string id;  
    string name;  
    int score;  
};
```

```
vector<Student> students;
```


五、主要程式架構與重要程式片段

2. 成績排序邏輯 (Sorting Logic)

使用<algorithm>中的std::sort 配合Lambda 函式，依據成績 (score) 進行由高至低的排序

```
void sortStudentsByScore(vector<Student>& students) {  
    if (students.empty()) {  
        cout << "目前沒有學生資料，無法排序。\\n";  
        return;  
    }  
  
    sort(students.begin(), students.end(), [](const Student& a, const Student& b) {  
        return a.score > b.score;  
    });  
  
    cout << "已依成績由高到低排序。\\n";  
    listStudents(students);  
}
```

五、主要程式架構與重要程式片段

3. 資料查詢與索引 (Search Logic)

走訪vector逐一比對學號，若相符則回傳該筆資料在陣列中的索引值（下標），若未找到則回傳 -1

```
int findStudentIndex(const vector<Student>& students, const string& id) {  
    for (size_t i = 0; i < students.size(); ++i) {  
        if (students[i].id == id) {  
            return static_cast<int>(i);  
        }  
    }  
    return -1;  
}
```

五、主要程式架構與重要程式片段

4. 使用者輸入防錯機制 (Input Validation)

利用 `cin.clear()` 重置錯誤旗標，並用 `cin.ignore()` 清除緩衝區殘留字元，搭配迴圈確保輸入數值落在 0~100 區間。

```
void clearInput() {  
    cin.clear();  
    cin.ignore(numeric_limits<streamsize>::max(), '\n');  
}
```

```
int readScore() {  
    int score;  
  
    while (true) {  
        cout << "請輸入成績：";  
        if (cin >> score && score >= 0 && score <= 100) {  
            clearInput();  
            return score;  
        }  
  
        cout << "成績輸入錯誤，請輸入 0 到 100 的整數。\\n";  
        clearInput();  
    }  
}
```

六、執行畫面或輸出結果

===== 學生資料管理系統 =====

1. 新增學生資料
2. 列出所有學生
3. 依成績排序
4. 查詢學生
5. 統計成績
0. 離開
請選擇功能：1
請輸入學號：114
請輸入姓名：小豪
請輸入成績：90
學生資料新增成功。

===== 學生資料管理系統 =====

1. 新增學生資料
2. 列出所有學生
3. 依成績排序
4. 查詢學生
5. 統計成績
0. 離開
請選擇功能：1
請輸入學號：115
請輸入姓名：邦邦
請輸入成績：40
學生資料新增成功。

===== 學生資料管理系統 =====

1. 新增學生資料
2. 列出所有學生
3. 依成績排序
4. 查詢學生
5. 統計成績
0. 離開
請選擇功能：1
請輸入學號：116
請輸入姓名：小夢
請輸入成績：70
學生資料新增成功。

===== 學生資料管理系統 =====

1. 新增學生資料
2. 列出所有學生
3. 依成績排序
4. 查詢學生
5. 統計成績
0. 離開
請選擇功能：2

學號	姓名	成績
114	小豪	90
115	邦邦	40
116	小夢	70

===== 學生資料管理系統 =====

1. 新增學生資料
2. 列出所有學生
3. 依成績排序
4. 查詢學生
5. 統計成績
0. 離開
請選擇功能：3
已依成績由高到低排序。

學號	姓名	成績
114	小豪	90
116	小夢	70
115	邦邦	40

===== 學生資料管理系統 =====

1. 新增學生資料
2. 列出所有學生
3. 依成績排序
4. 查詢學生
5. 統計成績
0. 離開
請選擇功能：4
請輸入要查詢的學號：115

學號	姓名	成績
115	邦邦	40

===== 學生資料管理系統 =====

1. 新增學生資料
2. 列出所有學生
3. 依成績排序
4. 查詢學生
5. 統計成績
0. 離開
請選擇功能：5
全班平均：66.67
最高分：90
最低分：40
及格人數：2
不及格人數：1

七、GitHub repo link與檔案結構說明

連結:<https://github.com/jhao-dev/Student-Information-Management-System-SIMS->

main.cpp

C++ 程式主檔，包含學生資料結構 (Student)、vector 容器管理、輸入防錯、成績排序、統計函式與極值樣板函式 (Template Function)。

README.md

專案說明文件。採用精煉段落整合專案背景、功能列表、編譯指令 (g++) 與跨平台執行方式。

report.pdf

自主學習成果報告，涵蓋學習主題、樣板與 STL 關聯、系統架構、執行畫面、GitHub 連結及反思檢討等等

八、遇到的問題與解決方式

在撰寫與除錯程式的過程中，主要遭遇了語法細節與環境設定的錯誤。首先是初期的編譯階段，因為誤用 C 語言編譯器（gcc）來建置 C++ 專案，導致 `std::cout` 等標準函式庫符號無法解析而報錯，經檢查後將 VS Code 的任務命令修正為 `g++` 才排除連結障礙。其次，在實作輸入邏輯時，發現混合使用 `cin >>` 與 `getline()` 會導致換行字元殘留在緩衝區，造成後續輸入直接跳過的問題；

為此實作了 `clearInput()` 函式，利用 `cin.clear()` 重置錯誤狀態並搭配 `cin.ignore()` 清除緩衝區。此外，自訂樣板函式（Template）時，也曾因型態不一致引發編譯錯誤，透過嚴格限制傳入參數型態與範圍檢查（如成績 `readScore` 限縮於 0 到 100）才確保了程式的穩健運行。

九、學習心得與未來可延伸方向

本次自主學習透過C++實作學生資料管理系統，讓我深刻體會到樣板與STL容器在泛型編程中的核心價值，不僅實現資料結構與具體型態的解耦，更大幅提升程式碼的復用性與開發效率。

展望未來，此系統架構具備多項可延伸的優化方向，包含將現有的動態陣列替換為以學號作為 Key 值的 map 容器，以常數時間複雜度優化查詢效率；導入物件導向設計（OOP）原則，將資料與操作封裝為類別來強化模組化；並結合檔案串流（fstream）實作資料讀寫，解決程式關閉後記憶體資料遺失的問題，進一步提升軟體的實用商業價值。